

Instructor-Led Lab: Insertion Sort Algorithm

Understanding and Implementing Insertion Sort in Python

Lab Overview

This instructor-led lab introduces you to the insertion sort algorithm, a fundamental sorting technique that builds a sorted array one element at a time. You will learn how the algorithm works conceptually, trace through examples by hand, and implement the algorithm in Python.

Learning Objectives:

- Understand the insertion sort algorithm and its mechanics
- Trace insertion sort execution on paper
- Implement insertion sort in Python
- Analyze the time and space complexity of insertion sort
- Compare insertion sort with other sorting algorithms

Duration: 75-90 minutes

Prerequisites: Basic Python programming, understanding of lists and loops

Part 1: Introduction to Insertion Sort (15 minutes)

What is Insertion Sort?

Insertion sort is a simple sorting algorithm that works by building a sorted portion of the list one element at a time. It's similar to how you might sort playing cards in your hand: you pick up cards one by one and insert each into its correct position among the cards you're already holding.

Key Concept: At any point during the algorithm, the left portion of the array is sorted, and we insert the next unsorted element into its correct position within that sorted portion.

Algorithm Steps

1. Start with the second element (index 1) - the first element is already "sorted"
2. Compare the current element with elements in the sorted portion (to its left)
3. Shift larger elements one position to the right
4. Insert the current element into its correct position
5. Repeat for all remaining elements

Demonstration: Card Sorting

Materials needed: 5-7 playing cards or index cards with numbers

Have students shuffle the cards and then sort them using the insertion sort method. Walk through each step:

- Hold the first card in your left hand (the "sorted" portion)
- Pick up the next card with your right hand

- Compare it with cards in your left hand from right to left
- Insert it in the correct position
- Repeat until all cards are sorted

Part 2: Tracing Insertion Sort by Hand (20 minutes)

Example Trace

Let's trace insertion sort on the array: [64, 25, 12, 22, 11]

Initial array: [64, 25, 12, 22, 11]

Pass 1 (i=1): Insert 25

Current element: 25

Compare with 64: $25 < 64$, so shift 64 right

Insert 25 at index 0

Result: [25, 64, 12, 22, 11]

Pass 2 (i=2): Insert 12

Current element: 12

Compare with 64: $12 < 64$, shift 64 right

Compare with 25: $12 < 25$, shift 25 right

Insert 12 at index 0

Result: [12, 25, 64, 22, 11]

Pass 3 (i=3): Insert 22

Current element: 22

Compare with 64: $22 < 64$, shift 64 right

Compare with 25: $22 < 25$, shift 25 right

Compare with 12: $22 > 12$, stop

Insert 22 at index 2

Result: [12, 22, 25, 64, 11]

Pass 4 (i=4): Insert 11

Current element: 11

Compare with 64, 25, 22, 12: all greater than 11

Shift all elements right

Insert 11 at index 0

Result: [11, 12, 22, 25, 64]

Final sorted array: [11, 12, 22, 25, 64]

Student Exercise: Paper Trace

Have students trace insertion sort on the following arrays:

Exercise 1: [5, 2, 4, 6, 1, 3]

Exercise 2: [38, 27, 43, 3, 9, 82, 10]

Students should show the array state after each pass. Walk around and check their work.

Part 3: Python Implementation (25 minutes)

Basic Implementation

Let's implement insertion sort step by step. Start by displaying the basic structure:

```
def insertion_sort(arr):
    # Start from the second element
    for i in range(1, len(arr)):
        # Store current element
        key = arr[i]

        # Move elements greater than key one position right
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert the key at its correct position
        arr[j + 1] = key
```

Code Walkthrough

Explain each part of the code:

- 1. Outer loop (for i in range(1, len(arr))):** Iterates through each element starting from index 1. Why start at 1? Because a single element (index 0) is already considered sorted.
- 2. Store the key:** We save the current element in a variable called 'key' because we're about to shift elements, potentially overwriting this position.
- 3. Inner while loop:** Starting from the position before the current element ($j = i - 1$), we move backward through the sorted portion, shifting elements that are greater than the key one position to the right.
- 4. Insert the key:** Once we find the correct position (when we encounter an element smaller than key or reach the beginning), we insert the key.

Enhanced Version with Visualization

Create a version that shows the sorting process:

```
def insertion_sort_verbose(arr):
    print(f"Initial array: {arr}")
    print()

    for i in range(1, len(arr)):
        key = arr[i]
        print(f"Pass {i}: Inserting {key}")

        j = i - 1
        while j >= 0 and arr[j] > key:
            print(f"  Shifting {arr[j]} to the right")
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key
```

```

        arr[j + 1] = key
        print(f"  Array after pass {i}: {arr}")
        print()

    print(f"Final sorted array: {arr}")

```

Testing the Implementation

Test your implementation with various inputs:

```

# Test cases
test1 = [64, 25, 12, 22, 11]
insertion_sort_verbose(test1)

test2 = [5, 2, 4, 6, 1, 3]
insertion_sort(test2)
print(test2)  # Should print [1, 2, 3, 4, 5, 6]

# Edge cases
test3 = [1]  # Single element
test4 = [1, 2, 3, 4, 5]  # Already sorted
test5 = [5, 4, 3, 2, 1]  # Reverse sorted

```

Part 4: Algorithm Analysis (15 minutes)

Time Complexity

Best Case: $O(n)$

Occurs when the array is already sorted. The inner while loop never executes because each element is already in its correct position. We only make one pass through the array.

Worst Case: $O(n^2)$

Occurs when the array is sorted in reverse order. For each element, we must compare it with all previously sorted elements and shift them all. This results in:

- Pass 1: 1 comparison
- Pass 2: 2 comparisons
- Pass 3: 3 comparisons
- ... and so on

Total: $1 + 2 + 3 + \dots + (n-1) = n(n-1)/2 = O(n^2)$

Average Case: $O(n^2)$

On average, each element needs to be compared with about half of the sorted portion, still resulting in quadratic time.

Space Complexity

$O(1)$ - Constant Space

Insertion sort is an in-place sorting algorithm. It only requires a constant amount of additional memory (the 'key' and 'j' variables), regardless of input size.

When to Use Insertion Sort

Advantages:

- Simple to understand and implement
- Efficient for small data sets
- Adaptive: $O(n)$ time when array is nearly sorted
- Stable: maintains relative order of equal elements
- In-place: requires only $O(1)$ additional memory
- Online: can sort a list as it receives it

Disadvantages:

- $O(n^2)$ time complexity makes it inefficient for large datasets
- Not suitable when you need guaranteed $O(n \log n)$ performance

Best Use Cases:

- Small arrays (typically fewer than 10-20 elements)
- Nearly sorted data
- As a subroutine in more complex algorithms (like Timsort)
- When simplicity is more important than efficiency

Part 5: Practice Exercises

Exercise 1: Descending Order

Modify the insertion sort function to sort in descending order instead of ascending order.

```
def insertion_sort_descending(arr):
    # Your code here
    pass
```

Hint: Change the comparison operator in the while loop condition.

Exercise 2: Counting Comparisons

Modify the insertion sort function to count and return the number of comparisons made during sorting.

```
def insertion_sort_count(arr):
    comparisons = 0
    # Your code here

    return comparisons
```

Exercise 3: Sorting Objects

Use insertion sort to sort a list of student dictionaries by their grade (GPA).

```
students = [
    {'name': 'Alice', 'gpa': 3.5},
    {'name': 'Bob', 'gpa': 3.9},
    {'name': 'Charlie', 'gpa': 3.2},
    {'name': 'Diana', 'gpa': 3.8}
]

def insertion_sort_students(arr):
    # Your code here
    pass
```

Hint: Access the GPA using `arr[j]['gpa']` when comparing.

Notes and Tips

Common Student Mistakes

- 1. Starting the outer loop at index 0:** Remind students that a single element is already sorted, so we start at index 1.
- 2. Forgetting to save the key:** Students often forget that shifting elements will overwrite the current value, so we must save it first.
- 3. Incorrect insertion position:** The final insertion is at $j + 1$, not j . This is because j was decremented one extra time in the while loop.
- 4. Off-by-one errors:** Watch for issues with $j \geq 0$ boundary condition.

Discussion Questions

- Why is insertion sort called a 'stable' sorting algorithm?
- In what real-world scenarios would insertion sort be preferable to quicksort or mergesort?
- How could you optimize insertion sort for nearly-sorted data?
- What's the difference between insertion sort and selection sort?

Lab Summary

In this lab, students learned:

- The conceptual basis of insertion sort (building a sorted portion one element at a time)
- How to trace insertion sort execution by hand
- How to implement insertion sort in Python
- The time and space complexity of the algorithm
- When insertion sort is appropriate to use
- How to modify the algorithm for different sorting criteria

Next Steps: Compare insertion sort with other sorting algorithms (selection sort, bubble sort, merge sort, quicksort) to understand the tradeoffs between different approaches and develop intuition for algorithm selection.